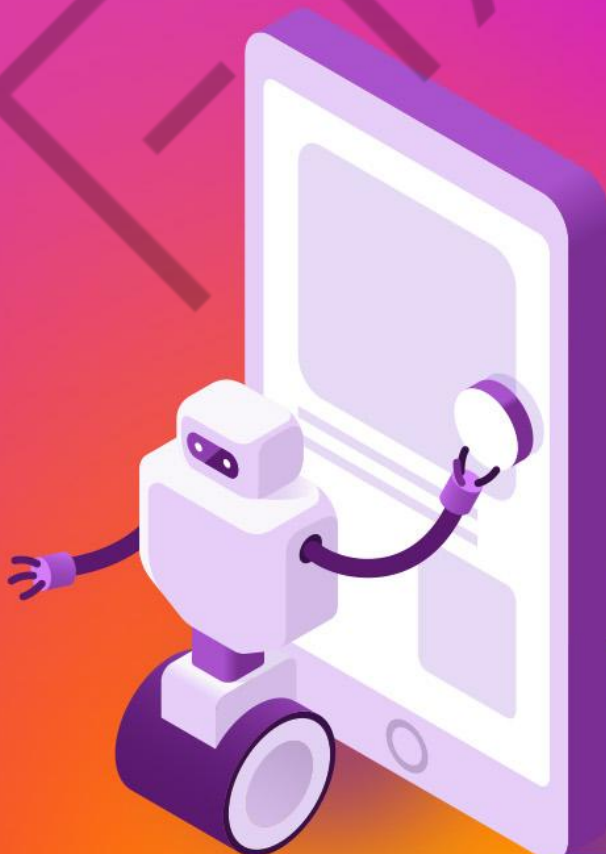


GESTÃO DE CLUSTERS,  
SEGURANÇA QUÂNTICA E  
COLABORAÇÃO MULTIAGENTE

# *ML EM ESCALA MÍNIMA: MODELOS OTIMIZADOS PARA MICROCONTROLADORES*



9

## LISTA DE FIGURAS

|                                                                                                                |    |
|----------------------------------------------------------------------------------------------------------------|----|
| Figura 1 - Estrutura do fluxo de inferência embarcada no ESP32.....                                            | 10 |
| Figura 2 - Representação do uso de janelas deslizantes com sobreposição no sinal de aceleração do eixo Z ..... | 15 |



## LISTA DE QUADROS

|                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------|----|
| Quadro 1 - Diferenças entre treinamento e inferência em sistemas de Machine Learning aplicados a IoT ..... | 8  |
| Quadro 2 - Features utilizadas como entrada do modelo.....                                                 | 14 |
| Quadro 3 - Organização lógica do firmware para inferência em tempo real no ESP32 .....                     | 20 |
| Quadro 4 - Etapas da aplicação do modelo no firmware.....                                                  | 29 |

EMANIP

## LISTA DE CÓDIGOS-FONTE

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Código-fonte 1 - Firmware do ESP32 para inferência em tempo real ..... | 28 |
|------------------------------------------------------------------------|----|

ESP32

## SUMÁRIO

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 1 ML EM ESCALA MÍNIMA: MODELOS OTIMIZADOS PARA MICROCONTROLADORES..... | 6  |
| 1.1 Fundamentos da Inferência em Microcontroladores .....              | 6  |
| 1.2 Conceito de inferência em dispositivos IoT .....                   | 6  |
| 1.3 Diferença entre treinamento e inferência.....                      | 7  |
| 1.4 Características de modelos adequados ao ESP32 .....                | 8  |
| 2 ESTRUTURA DO MODELO DE MACHINE LEARNING NO FIRMWARE .....            | 10 |
| 2.1 Representação do modelo como função matemática .....               | 11 |
| 2.2 Entrada do modelo: features e vetores de dados.....                | 13 |
| 2.3 Janela de dados e processamento contínuo .....                     | 14 |
| 3 IMPLEMENTAÇÃO DA INFERÊNCIA NO ESP32 .....                           | 17 |
| 3.1 Organização do firmware para inferência em tempo real .....        | 18 |
| 3.2 Cálculo das features em tempo real.....                            | 21 |
| 3.3 Aplicação do modelo e tomada de decisão .....                      | 23 |
| 3.4 Ação física baseada no resultado da inferência .....               | 29 |
| REFERÊNCIAS.....                                                       | 32 |

# 1 ML EM ESCALA MÍNIMA: MODELOS OTIMIZADOS PARA MICROCONTROLADORES

## 1.1 Fundamentos da Inferência em Microcontroladores

A inferência embarcada permite que o dispositivo tome decisões no próprio microcontrolador. O processamento ocorre no ponto onde o dado é gerado. Isso reduz a latência, diminui o tráfego de dados e mantém o sistema funcional sem conexão constante. Em projetos IoT, essa abordagem atende cenários que exigem resposta imediata e operação contínua.

Neste capítulo, a inferência é tratada como execução local de um modelo previamente definido. O foco está no que acontece durante a operação do dispositivo. O microcontrolador coleta dados do sensor, processa esses dados e produz uma decisão em tempo real. Não há treinamento no dispositivo. Há execução repetida de uma lógica matemática simples e previsível.

O ESP32 oferece recursos suficientes para esse tipo de tarefa. Ele executa operações aritméticas com estabilidade, gerencia buffers de dados e controla periféricos. Isso viabiliza a aplicação de modelos leves de classificação diretamente no firmware. O resultado é um sistema autônomo, com baixo consumo e resposta consistente.

A inferência embarcada exige organização clara do fluxo de dados. O firmware precisa garantir leitura periódica do sensor, armazenamento temporário das amostras e execução do modelo no instante correto. Cada etapa deve ser determinística. Essa previsibilidade facilita validação, depuração e manutenção do código.

Ao final desta seção, você entende o papel da inferência em sistemas IoT e o motivo de ela ser executada no ESP32. Nas próximas seções, o texto avança para a estrutura do modelo no firmware e para a implementação prática da inferência em tempo real.

## 1.2 Conceito de inferência em dispositivos IoT

Inferência em dispositivos IoT corresponde à aplicação de um modelo de Machine Learning já definido sobre dados coletados em tempo de execução. O

dispositivo executa o modelo diretamente no firmware e produz uma saída a cada ciclo de processamento.

Nesse processo, o modelo não sofre ajustes ou aprendizado. Seus parâmetros permanecem fixos durante a operação. O papel do dispositivo é receber dados do sensor, organizar essas informações no formato esperado e calcular o resultado da função do modelo.

A saída da inferência representa uma decisão objetiva. Ela pode indicar uma classe, um estado do sistema ou a ocorrência de um evento específico. Esse resultado é usado imediatamente pelo firmware para registrar informações ou acionar componentes físicos.

Em microcontroladores como o ESP32, a inferência é implementada por meio de operações matemáticas diretas. O código executa somas, multiplicações e comparações de forma repetitiva e previsível. Essa característica garante estabilidade, baixo custo computacional e execução contínua em aplicações IoT.

### **1.3 Diferença entre treinamento e inferência**

O ciclo de vida de um sistema de Machine Learning é composto por duas etapas distintas: treinamento e inferência. Cada etapa possui finalidade, requisitos computacionais e ambientes de execução diferentes. Essa distinção é determinante para a aplicação de modelos em dispositivos IoT.

O treinamento corresponde à fase em que o modelo é ajustado a partir de dados rotulados. Nessa etapa, algoritmos modificam seus parâmetros internos de forma iterativa com o objetivo de reduzir erros. Esse processo demanda alto poder de processamento, grande volume de memória e tempo elevado de execução, o que inviabiliza sua realização em microcontroladores.

A inferência ocorre após a definição final do modelo. Nessa etapa, os parâmetros permanecem fixos e o modelo passa a ser aplicado diretamente sobre novos dados coletados em tempo real. O dispositivo executa apenas operações matemáticas diretas para produzir uma decisão a cada ciclo de processamento.

O quadro a seguir apresenta uma síntese das principais diferenças entre treinamento e inferência no contexto de aplicações IoT.

| Aspecto analisado         | Treinamento                       | Inferência                     |
|---------------------------|-----------------------------------|--------------------------------|
| Finalidade                | Ajuste dos parâmetros do modelo   | Aplicação do modelo treinado   |
| Ambiente de execução      | Computador, servidor ou nuvem     | Dispositivo embarcado ou borda |
| Volume de dados           | Grande volume de dados históricos | Dados gerados em tempo real    |
| Custo computacional       | Elevado                           | Reduzido                       |
| Atualização de parâmetros | Presente                          | Ausente                        |
| Requisito de tempo real   | Não prioritário                   | Essencial                      |
| Execução no ESP32         | Não aplicável                     | Adequada                       |

Quadro 1 - Diferenças entre treinamento e inferência em sistemas de Machine Learning aplicados a IoT

Fonte: Goodfellow *et al.* (2016); Warden e Situnayake (2020), adaptado pelo autor (2026)

O treinamento e a inferência cumprem papéis distintos e bem definidos. Em aplicações IoT, essa separação garante que o esforço computacional fique concentrado fora do dispositivo, enquanto a tomada de decisão ocorre localmente. No ESP32, o firmware não executa aprendizado nem atualização de parâmetros. Ele apenas aplica o modelo previamente definido para classificar dados e gerar decisões em tempo real, respeitando as restrições de hardware do microcontrolador.

#### 1.4 Características de modelos adequados ao ESP32

Modelos executados no ESP32 precisam atender a restrições claras de hardware. O microcontrolador possui memória limitada, capacidade de processamento reduzida e operação em tempo real. Essas condições determinam quais tipos de modelos podem ser embarcados com estabilidade.

Modelos adequados ao ESP32 apresentam estrutura simples e número reduzido de parâmetros. A inferência deve ser realizada por meio de operações aritméticas diretas, como somas, multiplicações e comparações. Esse perfil garante tempo de execução previsível e baixo consumo de recursos.

O uso de modelos lineares ou de baixa complexidade é recomendado. Esses modelos permitem implementação direta no firmware, sem dependência de bibliotecas



externas ou aceleração por hardware. Além disso, facilitam a validação do comportamento do sistema e a análise de falhas durante a operação.

Outro fator relevante é o consumo de memória. O modelo, suas constantes e as estruturas de dados associadas devem caber integralmente na memória disponível do dispositivo. Isso inclui buffers de amostras, variáveis intermediárias e rotinas de processamento. Modelos compactos reduzem o risco de falhas e simplificam a manutenção do código.

Por fim, modelos adequados ao ESP32 devem ser determinísticos. Para um mesmo conjunto de entradas, o resultado da inferência precisa ser sempre o mesmo. Essa previsibilidade é essencial em aplicações IoT, em que decisões locais acionam componentes físicos e impactam diretamente o comportamento do sistema.

## 2 ESTRUTURA DO MODELO DE MACHINE LEARNING NO FIRMWARE

A implementação do modelo de Machine Learning no firmware do ESP32 parte da premissa de que a inteligência do sistema deve operar diretamente no dispositivo embarcado, utilizando apenas os dados coletados pelos sensores e os recursos computacionais disponíveis no microcontrolador. Nessa abordagem, todo o processo de inferência ocorre localmente, sem dependência de comunicação externa ou processamento remoto.

O modelo adotado no sistema é representado no firmware como uma função matemática explícita, composta por um conjunto fixo de parâmetros e aplicada diretamente sobre dados processados em tempo real. Essa representação elimina a necessidade de bibliotecas de aprendizado de máquina embarcadas e permite que o comportamento do modelo seja plenamente compreendido e controlado pelo desenvolvedor.

A figura a seguir apresenta o fluxo completo de aquisição, processamento e decisão implementado no firmware do ESP32.

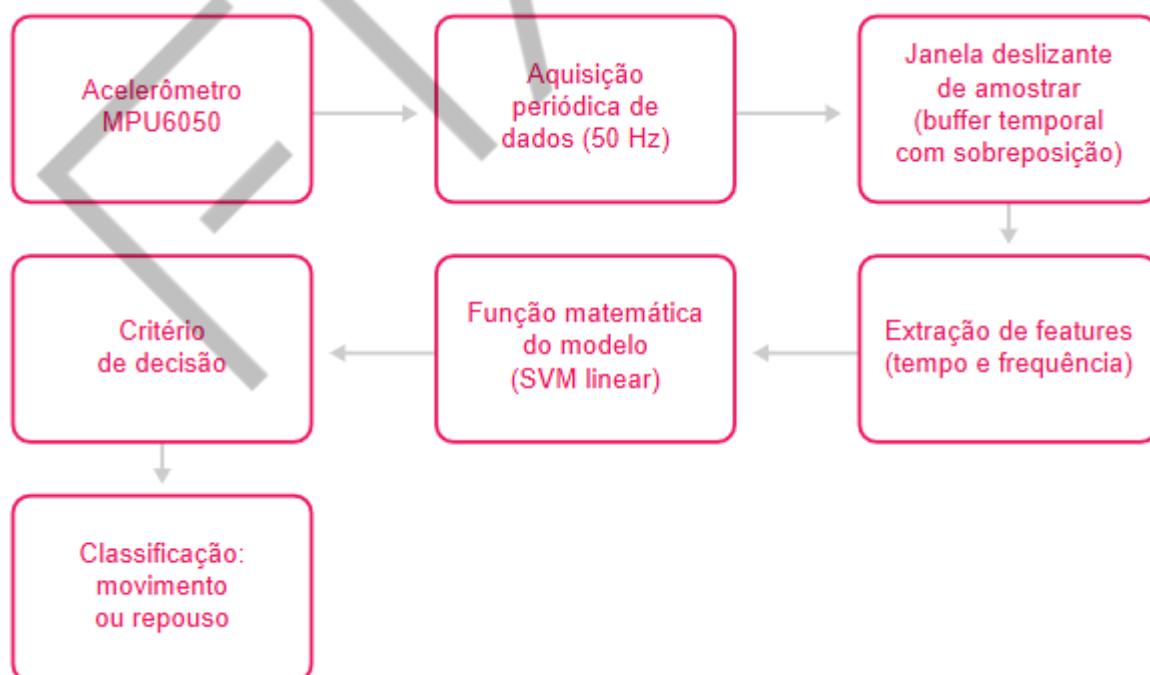


Figura 1 - Estrutura do fluxo de inferência embarcada no ESP32  
Fonte: Elaborado pelo autor (2026)

O diagrama ilustra as etapas desde a leitura periódica dos dados do acelerômetro MPU6050 até a classificação do estado do sistema, incluindo a organização das amostras em janelas deslizantes, a extração de features e a aplicação da função matemática do modelo.

Inicialmente, o acelerômetro MPU6050 fornece continuamente dados de aceleração ao microcontrolador. Essas leituras são realizadas de forma periódica, a uma taxa fixa de 50 Hz, garantindo uniformidade temporal na aquisição dos sinais. As amostras coletadas não são analisadas individualmente. Elas são armazenadas em um buffer temporal que forma uma janela deslizante de dados, permitindo que o sistema considere o comportamento do sinal ao longo de um intervalo definido.

Quando uma janela completa é formada, o firmware executa a extração das features no domínio do tempo e da frequência. Essas features sintetizam informações relevantes do sinal e constituem a entrada efetiva do modelo de Machine Learning. Em seguida, o vetor de features é aplicado à função matemática do modelo, implementada no código como uma combinação linear entre os parâmetros do modelo e os valores das features.

O resultado dessa função é avaliado por um critério de decisão simples, responsável por determinar a classe associada à janela analisada. Com base nesse critério, o sistema classifica o estado observado como movimento ou repouso, concluindo o ciclo de inferência.

Essa estrutura organiza claramente as responsabilidades de cada etapa do firmware e estabelece um fluxo contínuo de processamento, adequado para aplicações de IA embarcada que exigem resposta em tempo real e comportamento previsível. Nas subseções seguintes, cada componente desse fluxo é detalhado individualmente, iniciando pela representação do modelo como uma função matemática.

## **2.1 Representação do modelo como função matemática**

O núcleo de um sistema de inferência embarcada é o modelo matemático responsável por transformar um vetor de dados em uma decisão objetiva. No contexto deste capítulo, o modelo de Machine Learning é definido como uma função

matemática avaliada diretamente no firmware do microcontrolador, sem dependência de bibliotecas externas ou ambientes de execução complexos.

De forma geral, o modelo recebe como entrada um vetor de features extraído a partir de uma janela temporal do sinal do acelerômetro. Esse vetor representa, de maneira compacta, características relevantes do comportamento do sistema físico monitorado. A partir dessas informações, o modelo calcula um valor escalar que indica a posição do vetor de entrada em relação a uma fronteira de decisão previamente aprendida durante o treinamento.

No caso de um classificador linear, essa relação pode ser expressa pela função:

$$f(x) = w_0 \cdot x_0 + w_1 \cdot x_1 + \dots + w_n \cdot x_n + b$$

Em que  $x$  representa o vetor de features,  $w$  são os coeficientes associados a cada feature e  $b$  é o termo de bias. A avaliação dessa função envolve apenas multiplicações e somas, o que garante baixo custo computacional e tempo de execução constante, características fundamentais para aplicações em tempo real em sistemas embarcados.

Embora a formulação linear seja simples, máquinas de vetores de suporte permitem a utilização de funções de kernel capazes de modelar fronteiras de decisão não lineares. Entre os kernels mais conhecidos estão o kernel polinomial, definido por:

$$K(x, z) = (y \cdot x^T z + r)^d$$

E o kernel radial do tipo RBF, expresso por:

$$K(X, Z) = \exp(-\gamma \|x - z\|^2)$$

Essas funções realizam um mapeamento implícito dos dados para espaços de maior dimensionalidade, possibilitando a separação de padrões mais complexos. No entanto, a aplicação desses kernels implica maior custo computacional, necessidade de armazenar múltiplos vetores de suporte e dependência de operações matemáticas mais onerosas, como exponenciais e potências.

Em microcontroladores, essas características tornam a inferência não linear pouco adequada para execução contínua em tempo real. Por esse motivo, o modelo adotado neste capítulo é linear, priorizando previsibilidade temporal, estabilidade de execução e facilidade de implementação no firmware.

O critério de decisão é estabelecido diretamente a partir do sinal do valor  $f(x)$ . Quando o resultado é menor que zero, o vetor de features é associado à classe movimento. Quando o valor é maior ou igual a zero, a classificação corresponde ao estado de repouso. Esse limiar fixo decorre da própria formulação do classificador linear e elimina a necessidade de ajustes dinâmicos ou normalizações adicionais durante a inferência.

Essa abordagem oferece duas vantagens centrais no contexto embarcado. A primeira é a previsibilidade computacional, uma vez que o tempo de inferência é constante e independe do conteúdo do sinal. A segunda é a interpretabilidade, pois o valor escalar  $f(x)$  pode ser monitorado durante a execução, permitindo análise do comportamento do modelo e verificação da robustez da decisão.

A partir dessa definição matemática e do critério de decisão associado, o modelo torna-se diretamente integrável ao firmware do ESP32, servindo como base para as etapas de entrada de dados, organização das janelas temporais e execução contínua da inferência, discutidas nas subseções seguintes.

## **2.2 Entrada do modelo: features e vetores de dados**

A entrada do modelo de Machine Learning no firmware não é composta por amostras individuais do acelerômetro, mas por um vetor de features extraídas a partir de uma janela temporal de dados. Essa estratégia permite representar o comportamento do sinal ao longo do tempo de forma compacta e informativa, reduzindo a sensibilidade a ruídos instantâneos e variações pontuais.

No sistema implementado, o acelerômetro fornece amostras de aceleração ao longo do eixo Z, coletadas de forma periódica. Essas amostras são armazenadas em um buffer temporal que compõe uma janela deslizante com tamanho fixo. A cada janela completa, o firmware calcula um conjunto de features que descrevem estatisticamente e espectralmente o sinal observado naquele intervalo.

O vetor de entrada do modelo é formado por sete features, todas derivadas do eixo Z do acelerômetro. Essas features representam diferentes aspectos do sinal e foram selecionadas por sua relevância para a discriminação entre os estados analisados. O quadro a seguir mostra as features utilizadas, e suas respectivas funções:

| Feature   | Domínio    | Descrição                      | Intuição física                   |
|-----------|------------|--------------------------------|-----------------------------------|
| az_std    | Tempo      | Desvio padrão do sinal         | Indica variabilidade do movimento |
| az_ptp    | Tempo      | Pico a pico da aceleração      | Amplitude total do movimento      |
| az_fft_5  | Frequência | Magnitude da 5ª componente FFT | Presença de padrão periódico      |
| az_max    | Tempo      | Valor máximo do sinal          | Picos de aceleração               |
| az_rms    | Tempo      | Valor RMS                      | Energia média do movimento        |
| az_energy | Tempo      | Soma dos quadrados             | Intensidade global do sinal       |
| az_min    | Tempo      | Valor mínimo do sinal          | Vales de aceleração               |

Quadro 2 - Features utilizadas como entrada do modelo

Fonte: Elaborado pelo autor (2026)

Essas sete medidas compõem o vetor  $x$  aplicado ao modelo matemático. A ordem das features no vetor é fixa e deve ser preservada no firmware, pois cada posição corresponde diretamente a um coeficiente específico do modelo.

Ao utilizar apenas features previamente calculadas, o modelo passa a operar sobre uma representação de dimensão reduzida e semanticamente significativa do sinal. Isso simplifica o processamento no microcontrolador e garante que a inferência seja realizada de forma eficiente e consistente em tempo real.

Na próxima subseção, é apresentada a forma como a saída do modelo é interpretada e como o critério de decisão é aplicado para classificar cada janela analisada.

## 2.3 Janela de dados e processamento contínuo

Em sistemas embarcados com aquisição contínua de sinais, como aqueles baseados em acelerômetros para detecção de movimento, o processamento de amostras isoladas não é suficiente para capturar padrões temporais relevantes. Para contornar essa limitação, o modelo de Machine Learning embarcado opera sobre janelas de dados, isto é, blocos consecutivos de amostras que representam o comportamento do sinal ao longo de um intervalo de tempo.

No sistema proposto, os dados do acelerômetro MPU6050 são adquiridos de forma periódica a uma frequência de 50 Hz, resultando em uma nova amostra a cada 20 ms. Essas amostras são armazenadas em um buffer temporal de tamanho fixo,

formando janelas compostas por 20 amostras consecutivas, o que corresponde a aproximadamente 400 ms de sinal.

Para permitir processamento contínuo e reduzir o tempo de resposta do sistema, é empregada a técnica de janela deslizante com sobreposição. Nessa abordagem, após o processamento de uma janela completa, apenas parte das amostras mais antigas é descartada, enquanto as amostras mais recentes são reaproveitadas na próxima janela.

No firmware, essa lógica é implementada com uma sobreposição de 50%, ou seja, metade das amostras da janela anterior é reutilizada na janela seguinte. Essa estratégia reduz a perda de informação temporal e aumenta a probabilidade de detecção de eventos que ocorram na transição entre janelas consecutivas.

A próxima figura ilustra esse conceito de forma visual, mostrando o sinal de aceleração no eixo Z ao longo do tempo, as janelas deslizantes sucessivas, a região de sobreposição entre elas e o intervalo utilizado para rotulagem durante a fase de construção do dataset.

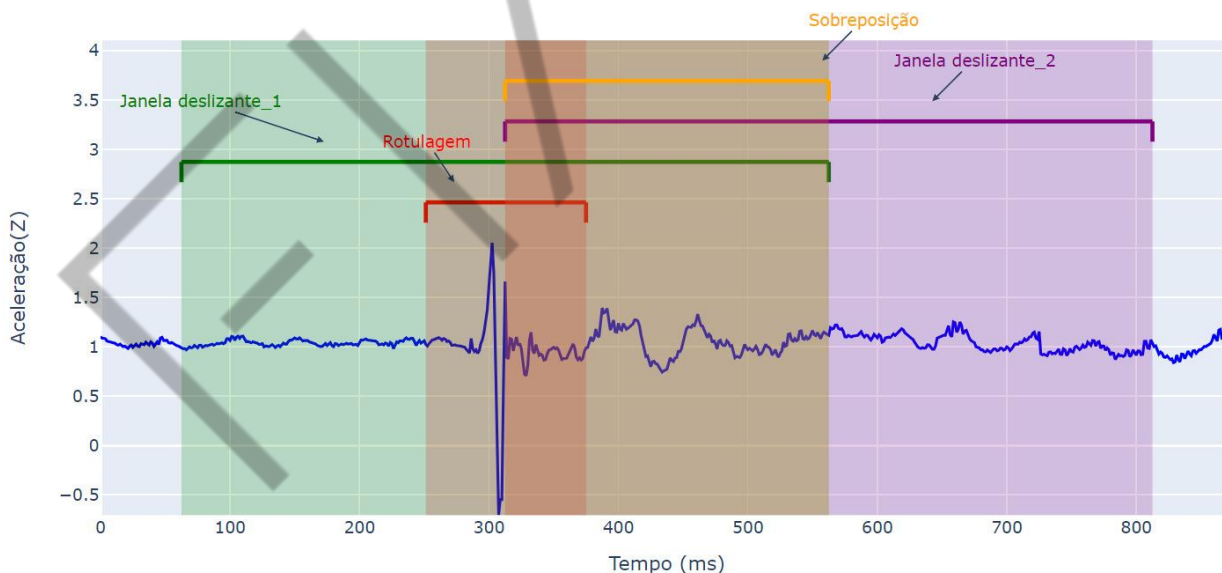


Figura 2 - Representação do uso de janelas deslizantes com sobreposição no sinal de aceleração do eixo Z

Fonte: Elaborado pelo autor (2026)

O sinal contínuo é segmentado em janelas temporais parcialmente sobrepostas, permitindo processamento contínuo e detecção mais robusta de eventos de movimento.

Sempre que uma janela é completamente preenchida, o firmware executa, de forma sequencial, as seguintes etapas:

1. Adquirir amostras do acelerômetro de forma periódica, respeitando a taxa de amostragem definida.
2. Armazenar as amostras em um buffer temporal com tamanho fixo.
3. Atualizar o buffer a cada nova amostra, aplicando a sobreposição entre janelas consecutivas.
4. Extrair as features no domínio do tempo e da frequência a partir da janela completa.
5. Calcular a saída do modelo por meio da função matemática linear.
6. Aplicar o critério de decisão para classificar o estado atual.
7. Repetir o processo continuamente, permitindo a detecção em tempo real.

Após a conclusão dessas etapas, o buffer é parcialmente deslocado, preservando as amostras correspondentes à região de sobreposição, e o sistema retorna imediatamente à etapa de aquisição de dados. Esse ciclo ocorre continuamente, permitindo que o microcontrolador opere em tempo real, sem necessidade de armazenamento prolongado dos dados brutos.

Essa abordagem é particularmente adequada para dispositivos embarcados com restrições de memória e processamento, pois combina baixo custo computacional, baixa latência e capacidade de detecção contínua, tornando viável a aplicação de modelos de Machine Learning diretamente no firmware.



### 3 IMPLEMENTAÇÃO DA INFERÊNCIA NO ESP32

A implementação de inferência de modelos de Machine Learning em microcontroladores representa um dos principais desafios práticos da Inteligência Artificial aplicada a sistemas embarcados. Diferentemente de ambientes computacionais tradicionais, como servidores ou computadores pessoais, dispositivos como o ESP32 operam sob severas restrições de processamento, memória e consumo energético. Dessa forma, a transposição de um modelo treinado em ambiente offline para execução direta no firmware exige decisões arquiteturais cuidadosas.

Neste contexto, a inferência não deve ser entendida apenas como a aplicação de um modelo matemático, mas como parte de um pipeline contínuo de aquisição, processamento e tomada de decisão, executado em tempo real. Cada etapa desse pipeline precisa ser explicitamente implementada no firmware, desde a coleta periódica dos dados do sensor até a geração de uma ação física baseada no resultado da classificação.

O ESP32 foi escolhido como plataforma de execução por oferecer um equilíbrio adequado entre capacidade computacional, suporte a periféricos e recursos de temporização, sendo amplamente utilizado em aplicações de IoT e sistemas ciberfísicos. No entanto, mesmo com recursos superiores a microcontroladores mais simples, a execução de inferência em tempo real requer a adoção de modelos leves, operações matemáticas eficientes e estruturas de dados enxutas.

A abordagem adotada neste projeto baseia-se em um modelo de classificação linear, cuja inferência pode ser expressa como uma função matemática simples. Essa escolha não é casual: modelos lineares apresentam baixa complexidade computacional, comportamento previsível e fácil integração ao código embarcado, tornando-se particularmente adequados para execução contínua em microcontroladores.

Além do modelo em si, a implementação da inferência envolve a definição de uma estratégia de processamento temporal dos dados. Em vez de realizar decisões pontuais a partir de amostras isoladas, o firmware opera sobre janelas de dados, permitindo capturar padrões temporais relevantes do sinal do acelerômetro. Essa

estratégia é fundamental para diferenciar estados como movimento e repouso de forma robusta, reduzindo a sensibilidade a ruídos instantâneos.

Outro aspecto central da implementação é a separação clara entre tarefas de aquisição de dados e tarefas de processamento. A coleta das amostras é realizada de forma periódica, respeitando uma taxa de amostragem fixa, enquanto o processamento das janelas e a inferência ocorrem no laço principal do programa. Essa organização permite manter a regularidade da amostragem sem comprometer a execução do modelo.

Por fim, a inferência embarcada não se encerra na classificação em si. O resultado do modelo deve ser interpretado pelo firmware para gerar uma ação concreta no mundo físico, como a ativação de um atuador, o envio de uma mensagem ou o acionamento de uma lógica de controle. Dessa forma, a implementação da inferência conecta diretamente o domínio digital do Machine Learning ao domínio físico do sistema embarcado.

As seções a seguir detalham cada uma dessas etapas, apresentando a organização do firmware, o cálculo das features em tempo real, a aplicação do modelo matemático e a definição das ações físicas associadas ao resultado da inferência.

### **3.1 Organização do firmware para inferência em tempo real**

A organização do firmware é um fator determinante para o funcionamento correto da inferência em tempo real em sistemas embarcados. Diferentemente de aplicações convencionais, nas quais as tarefas podem ser executadas de forma sequencial e sem restrições temporais rigorosas, a inferência embarcada impõe a necessidade de uma arquitetura capaz de conciliar aquisição periódica de dados, processamento contínuo e tomada de decisão, sem comprometer a estabilidade e a previsibilidade do sistema.

No ESP32, essa organização é construída a partir da separação clara entre responsabilidades funcionais bem definidas. Em especial, o firmware distingue explicitamente os processos de aquisição de dados do sensor e de processamento das janelas de dados com aplicação do modelo de Machine Learning. Essa separação é fundamental para garantir que a coleta das amostras ocorra com periodicidade

constante, independentemente do tempo necessário para o cálculo das features ou para a execução da inferência.

A aquisição dos dados do acelerômetro é realizada por meio de um mecanismo de temporização periódica. Um temporizador interno do ESP32 é configurado para executar uma rotina de coleta em intervalos fixos, assegurando uma taxa de amostragem estável. A cada disparo do temporizador, uma nova leitura do acelerômetro é obtida e armazenada em um buffer de memória. Esse processo ocorre de forma automática e independente do fluxo principal do programa, característica essencial para aplicações em tempo real.

Enquanto isso, o laço principal do firmware permanece responsável pelo processamento lógico do sistema. Nesse laço, o código verifica continuamente se uma janela completa de amostras já foi preenchida. Somente quando essa condição é satisfeita o processamento das features e a aplicação do modelo de Machine Learning são executados. Essa estratégia evita cálculos desnecessários, reduz o consumo computacional e garante que a inferência seja realizada apenas quando os dados disponíveis representam adequadamente o comportamento do sinal.

O uso de buffers temporais desempenha um papel central nessa organização. As amostras coletadas pelo temporizador são armazenadas sequencialmente até que o tamanho da janela seja atingido. Após o processamento, parte dessas amostras é mantida no buffer para compor a próxima janela, caracterizando a estratégia de janela deslizante com sobreposição. Essa abordagem permite que o sistema realize inferências de forma contínua, preservando a sensibilidade a transições entre estados, como a passagem de repouso para movimento.

Outro aspecto relevante da organização do firmware está relacionado ao controle de concorrência entre tarefas. Como a rotina de aquisição de dados e o processamento da inferência podem acessar simultaneamente o mesmo buffer de memória, torna-se necessário empregar mecanismos de sincronização para evitar inconsistências. No ESP32, isso é realizado por meio de seções críticas, que garantem acesso seguro aos dados compartilhados sem interromper o funcionamento do sistema ou introduzir atrasos indesejados.

Além disso, a lógica de inferência é estruturada de forma modular. As funções responsáveis pela aquisição dos dados, pelo cálculo das features, pela avaliação da

função matemática do modelo e pela tomada de decisão são implementadas de maneira independente. Essa organização modular facilita a leitura do código, a depuração e futuras extensões do sistema, como a substituição do modelo de classificação ou a inclusão de novas variáveis de entrada.

O quadro a seguir apresenta uma visão sintética da organização lógica do firmware para inferência em tempo real, destacando os principais blocos funcionais, suas responsabilidades e a frequência com que são executados no sistema embarcado.

| Bloco do firmware     | Responsabilidade principal                                 | Frequência de execução  |
|-----------------------|------------------------------------------------------------|-------------------------|
| Inicialização (setup) | Configurar sensores, temporizadores e parâmetros do modelo | Executado uma única vez |
| Aquisição de dados    | Ler aceleração do MPU6050 e armazenar no buffer            | Periódica (50 Hz)       |
| Buffer temporal       | Armazenar janelas de amostras com sobreposição             | Contínua                |
| Extração de features  | Calcular estatísticas no tempo e na frequência             | A cada janela completa  |
| Inferência            | Aplicar a função matemática do modelo SVM linear           | A cada janela           |
| Tomada de decisão     | Avaliar o critério de decisão do modelo                    | A cada inferência       |
| Ação física           | Executar resposta física ou sinalização                    | Sob condição            |

Quadro 3 - Organização lógica do firmware para inferência em tempo real no ESP32  
Fonte: Elaborado pelo autor (2026)

Por fim, a organização do firmware considera explicitamente o comportamento em tempo real do sistema. Após cada decisão de movimento ou repouso, o firmware pode executar uma ação física específica e, em seguida, preparar o sistema para a próxima janela de dados. Esse ciclo se repete continuamente enquanto o dispositivo estiver em operação, caracterizando um sistema reativo, capaz de responder dinamicamente às mudanças do ambiente físico.

Essa estrutura serve como base para as etapas seguintes de cálculo das features em tempo real, aplicação do modelo de Machine Learning e execução das ações físicas associadas ao resultado da classificação, garantindo eficiência, previsibilidade e alinhamento com as restrições do hardware embarcado.

### 3.2 Cálculo das features em tempo real

O cálculo das features em tempo real constitui uma das etapas mais críticas da inferência embarcada, pois é nesse momento que o sinal bruto proveniente do sensor é transformado em um conjunto compacto de informações relevantes para o modelo de Machine Learning. Em sistemas embarcados, essa etapa deve ser cuidadosamente projetada para equilibrar expressividade estatística, custo computacional e restrições de memória.

No contexto deste sistema, as features são calculadas diretamente no firmware do ESP32, a partir das amostras do acelerômetro armazenadas em uma janela temporal. Cada janela representa um segmento do sinal no domínio do tempo, contendo informações suficientes para caracterizar o comportamento dinâmico do sistema físico durante aquele intervalo.

A escolha das features considera dois aspectos fundamentais. O primeiro é a capacidade de discriminar os estados de interesse, neste caso movimento e repouso. O segundo é a viabilidade de cálculo em tempo real, evitando operações excessivamente complexas ou dependentes de grandes estruturas de dados.

O cálculo das features ocorre somente quando uma janela completa de amostras está disponível no buffer. Essa janela é composta por um número fixo de amostras adquiridas a uma taxa constante, garantindo coerência temporal entre os dados analisados. Ao atingir o tamanho da janela, o firmware interrompe momentaneamente a aquisição para copiar os dados e iniciar o processamento estatístico.

Esse processamento é realizado sobre uma cópia local do buffer, o que evita conflitos com a rotina de aquisição periódica. Dessa forma, o sistema mantém a estabilidade da taxa de amostragem e assegura que as operações matemáticas não interfiram na coleta de novos dados.

As features no domínio do tempo são responsáveis por capturar características estatísticas diretamente relacionadas à amplitude, variabilidade e energia do sinal. Essas medidas são computacionalmente eficientes e altamente informativas para a identificação de padrões de movimento.

Entre as estatísticas calculadas estão o valor médio, o desvio padrão, os valores mínimo e máximo, o valor pico a pico, o valor RMS e a energia total do sinal. Essas grandezas permitem quantificar desde pequenas oscilações associadas ao ruído até variações mais intensas causadas por movimentos reais.

O cálculo dessas features envolve apenas operações aritméticas básicas, como somas, subtrações, multiplicações e raízes quadradas, o que as torna particularmente adequadas para execução em microcontroladores com recursos limitados.

Além das estatísticas temporais, o sistema também extrai informações no domínio da frequência. Para isso, é aplicada uma transformada discreta de Fourier simplificada sobre os dados da janela. Em vez de calcular o espectro completo, o firmware estima apenas a magnitude de componentes específicas da transformada.

Essa abordagem reduz significativamente o custo computacional e permite capturar padrões periódicos associados ao movimento, como vibrações ou oscilações repetitivas. A seleção de componentes específicas da transformada está alinhada ao processo de seleção de features realizado durante o treinamento do modelo, garantindo consistência entre a fase offline e a inferência embarcada.

A combinação de features temporais e espectrais resulta em um vetor de entrada capaz de representar tanto a intensidade quanto a estrutura dinâmica do sinal, aumentando a robustez da classificação.

No firmware, o cálculo das features é implementado de forma modular. Cada feature é obtida por meio de uma função dedicada, responsável por operar sobre o vetor de amostras da janela. Essa organização facilita a leitura do código, a validação dos resultados e possíveis ajustes futuros, como a inclusão ou remoção de variáveis.

As funções são chamadas sequencialmente, e os valores calculados são armazenados em variáveis escalares que compõem o vetor de entrada do modelo. Essa estratégia evita o uso de estruturas dinâmicas e reduz o consumo de memória, aspecto essencial em sistemas embarcados.

Outro ponto importante é que todas as features utilizadas na inferência são calculadas exclusivamente a partir de dados disponíveis no próprio dispositivo, sem qualquer dependência de processamento externo. Isso caracteriza efetivamente a execução de IA na borda, com baixa latência e autonomia operacional.

Como o cálculo das features ocorre em paralelo à aquisição contínua de dados, o firmware utiliza mecanismos de sincronização para garantir consistência no acesso às amostras. As seções críticas asseguram que os dados não sejam alterados durante o processamento, evitando resultados inconsistentes ou instabilidades no sistema.

Do ponto de vista computacional, o cálculo das features é otimizado para execução em tempo real. O tamanho reduzido da janela, aliado à seleção criteriosa das variáveis, permite que todo o processamento seja concluído dentro do intervalo entre janelas consecutivas, preservando o comportamento reativo do sistema.

Ao final dessa etapa, o firmware dispõe de um vetor de features completamente calculado e pronto para ser utilizado na aplicação da função matemática do modelo de Machine Learning. Essa transição marca o ponto em que o sistema deixa de operar no domínio do sinal bruto e passa a atuar no espaço de decisão do modelo, tema abordado na subseção seguinte.

### 3.3 Aplicação do modelo e tomada de decisão

A aplicação do modelo de Machine Learning no ESP32 corresponde ao momento em que o sistema transforma as features extraídas do sinal em uma decisão concreta sobre o estado do sistema físico. Diferentemente do treinamento realizado em ambiente computacional, essa etapa ocorre integralmente no dispositivo embarcado, em tempo real e sob restrições de processamento e memória.

O modelo utilizado neste sistema é um classificador linear, cuja decisão pode ser representada matematicamente por uma função do tipo:

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Em que o vetor  $x$  corresponde às features calculadas a partir da janela de dados,  $w$  representa os coeficientes aprendidos durante o treinamento e  $b$  é o termo de bias do modelo. No firmware, esses coeficientes são armazenados como constantes, permitindo que a inferência seja realizada por meio de operações aritméticas simples.

Após o cálculo das features no laço principal do programa, o firmware avalia diretamente essa função matemática. Cada feature é multiplicada pelo respectivo coeficiente do modelo e, ao final, o termo de bias é somado ao resultado. O valor

escalar obtido, denominado saída do modelo, é então comparado a um limiar de decisão previamente definido.

No caso de um SVM linear, esse limiar corresponde ao valor zero. Assim, o critério de decisão é simples e eficiente: valores negativos da função indicam uma classe, enquanto valores positivos indicam a outra. Essa característica torna o modelo especialmente adequado para execução embarcada, pois elimina a necessidade de funções complexas ou estruturas adicionais de decisão.

A implementação completa dessa lógica no ESP32 é apresentada no código-fonte a seguir, que integra a aquisição periódica de dados, o cálculo das features, a aplicação do modelo e o critério de decisão em um único firmware coeso.

```
#include <Wire.h>
#include <MPU6050.h>
#include "esp_timer.h"
#include <math.h>

#define ACCEL_SCALE_FACTOR 8192.0f
#define SAMPLE_PERIOD_US 20000
#define WINDOW_SIZE 20
#define OVERLAP_SIZE 10

static const float W[7] = {
    -0.00165011f,
    -0.00710379f,
    -0.01440110f,
    -0.00343501f,
    -0.00081765f,
    -0.04453173f,
    0.00366878f
};

static const float B = 1.8944849323f;

MPU6050 mpu;

static float az_buf[WINDOW_SIZE];
static volatile int idx = 0;
static volatile bool janela_pronta = false;

static portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

static esp_timer_handle_t timer_coleta;

// -----
// Funções de features (az)
```



```
// -----

float calc_min(const float* v, int n) {
    float mn = v[0];
    for (int i = 1; i < n; i++) if (v[i] < mn) mn = v[i];
    return mn;
}

float calc_max(const float* v, int n) {
    float mx = v[0];
    for (int i = 1; i < n; i++) if (v[i] > mx) mx = v[i];
    return mx;
}

float calc_mean(const float* v, int n) {
    float s = 0.0f;
    for (int i = 0; i < n; i++) s += v[i];
    return s / (float)n;
}

float calc_std_pop(const float* v, int n) {
    float m = calc_mean(v, n);
    float s2 = 0.0f;
    for (int i = 0; i < n; i++) {
        float d = v[i] - m;
        s2 += d * d;
    }
    return sqrtf(s2 / (float)n);
}

float calc_rms(const float* v, int n) {
    float s = 0.0f;
    for (int i = 0; i < n; i++) s += v[i] * v[i];
    return sqrtf(s / (float)n);
}

float calc_energy_sum(const float* v, int n) {
    float s = 0.0f;
    for (int i = 0; i < n; i++) s += v[i] * v[i];
    return s;
}

float calc_ptp(const float* v, int n) {
    return calc_max(v, n) - calc_min(v, n);
}

float calc_dft_mag_k(const float* v, int n, int k) {
    float re = 0.0f, im = 0.0f;
    for (int i = 0; i < n; i++) {
        float a = 2.0f * PI * (float)k * (float)i / (float)n;
        re += v[i] * cosf(a);
    }
}
```

```

        im -= v[i] * sinf(a);
    }
    return sqrtf(re * re + im * im);
}

// -----
// Controle de buffer
// -----

void reset_buffer() {
    portENTER_CRITICAL(&mux);
    for (int i = 0; i < WINDOW_SIZE; i++) az_buf[i] = 0.0f;
    idx = 0;
    janela_pronta = false;
    portEXIT_CRITICAL(&mux);
}

void aplicar_overlap() {
    portENTER_CRITICAL(&mux);
    for (int i = 0; i < OVERLAP_SIZE; i++) {
        az_buf[i] = az_buf[i + (WINDOW_SIZE - OVERLAP_SIZE)];
    }
    idx = OVERLAP_SIZE;
    janela_pronta = false;
    portEXIT_CRITICAL(&mux);
}

// -----
// Coleta periódica (timer)
// -----

void coletar(void* arg) {
    int16_t ax_raw, ay_raw, az_raw;
    mpu.getAcceleration(&ax_raw, &ay_raw, &az_raw);

    portENTER_CRITICAL_ISR(&mux);

    if (idx < WINDOW_SIZE) {
        az_buf[idx] = (float)az_raw / ACCEL_SCALE_FACTOR;
        idx++;
        if (idx >= WINDOW_SIZE) janela_pronta = true;
    }

    portEXIT_CRITICAL_ISR(&mux);
}

// -----
// Setup
// -----

void setup() {
    Serial.begin(115200);

```

```

Wire.begin();

mpu.initialize();
if (!mpu.testConnection()) {
    Serial.println("MPU6050 nao conectada!");
    while (1) { }
}

mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_4);
Serial.println("MPU6050          conectada.          Aguardando
dados...");

reset_buffer();

const esp_timer_create_args_t args = {
    .callback = &coletar,
    .arg = NULL,
    .dispatch_method = ESP_TIMER_TASK,
    .name = "coleta_50hz"
};

esp_timer_create(&args, &timer_coleta);
esp_timer_start_periodic(timer_coleta,
SAMPLE_PERIOD_US);
}

// -----
// Loop
// -----

void loop() {

    if (!janela_pronta) return;

    float az[WINDOW_SIZE];

    portENTER_CRITICAL(&mux);
    for (int i = 0; i < WINDOW_SIZE; i++) az[i] = az_buf[i];
    janela_pronta = false; // evita reprocessar a mesma
janela
    portEXIT_CRITICAL(&mux);

    float az_std      = calc_std_pop(az, WINDOW_SIZE);
    float az_ptp      = calc_ptp(az, WINDOW_SIZE);
    float az_fft_5    = calc_dft_mag_k(az, WINDOW_SIZE, 5);
    float az_max       = calc_max(az, WINDOW_SIZE);
    float az_rms       = calc_rms(az, WINDOW_SIZE);
    float az_energy    = calc_energy_sum(az, WINDOW_SIZE);
    float az_min       = calc_min(az, WINDOW_SIZE);

```

```

float fx = B
    + W[0] * az_std
    + W[1] * az_ptp
    + W[2] * az_fft_5
    + W[3] * az_max
    + W[4] * az_rms
    + W[5] * az_energy
    + W[6] * az_min;

if (fx < 0.0f) {

    esp_timer_stop(timer_coleta);

    Serial.println();
    Serial.println(">>> Movimento reconhecido!");
    Serial.printf("az_std      = %.6f\n", az_std);
    Serial.printf("az_ptp      = %.6f\n", az_ptp);
    Serial.printf("az_fft_5    = %.6f\n", az_fft_5);
    Serial.printf("az_max      = %.6f\n", az_max);
    Serial.printf("az_rms      = %.6f\n", az_rms);
    Serial.printf("az_energy   = %.6f\n", az_energy);
    Serial.printf("az_min      = %.6f\n", az_min);
    Serial.printf("f(x)        = %.6f\n", fx);

    delay(1000);

    reset_buffer();
    esp_timer_start_periodic(timer_coleta,
SAMPLE_PERIOD_US);
    return;
}

aplicar_overlap();
}

```

Código-fonte 1 - Firmware do ESP32 para inferência em tempo real  
 Fonte: Elaborado pelo autor (2026)

No código apresentado, os coeficientes do modelo e o termo de bias são definidos como constantes globais, garantindo acesso rápido e evitando o uso de memória dinâmica. Após a identificação de que uma janela completa de dados está disponível, o firmware calcula as features selecionadas e avalia a função do modelo de forma determinística.

Quando o critério de decisão indica a presença de movimento, o sistema interrompe temporariamente a aquisição de dados, registra o evento e executa a ação associada. Em seguida, o buffer é reinicializado e o temporizador de coleta é

reativado, permitindo que o sistema retorne imediatamente ao monitoramento contínuo.

De forma resumida, a aplicação do modelo no firmware pode ser compreendida como uma sequência lógica bem definida de etapas, desde a recepção das features até a execução da ação correspondente. Essas etapas estão sintetizadas no próximo quadro, que apresenta uma visão estruturada do processo de inferência embarcada no ESP32.

| <b>Etapas</b>            | <b>Descrição</b>                                             |
|--------------------------|--------------------------------------------------------------|
| Receber features         | Utilizar as features calculadas a partir da janela de dados. |
| Avaliar função do modelo | Calcular a combinação linear entre features, pesos e bias.   |
| Comparar com limiar      | Verificar o sinal da função $f(x)$ .                         |
| Tomar decisão            | Classificar como movimento ou repouso.                       |
| Executar ação            | Acionar resposta física e preparar próxima janela.           |

Quadro 4 - Etapas da aplicação do modelo no firmware  
Fonte: Elaborado pelo autor (2026)

A organização dessas etapas evidencia que a inferência embarcada não se resume à execução de um cálculo matemático isolado, mas sim a um processo contínuo de percepção, decisão e reação. Ao integrar diretamente o modelo de Machine Learning ao firmware, o ESP32 passa a operar como um sistema inteligente autônomo, capaz de interpretar sinais do ambiente físico e responder a eles em tempo real.

Essa abordagem estabelece a base para a próxima etapa do sistema, na qual a decisão produzida pelo modelo é convertida em uma ação física concreta, fechando o ciclo entre sensoriamento, processamento e atuação no mundo real.

**3.4 Ação física baseada no resultado da inferência**

A ação física baseada no resultado da inferência representa a etapa final do ciclo de funcionamento do sistema embarcado inteligente. Após a aquisição dos dados, o cálculo das features e a aplicação do modelo de Machine Learning, o resultado da classificação precisa ser convertido em uma resposta concreta do dispositivo, caracterizando a interação entre o sistema digital e o ambiente físico.

No contexto da inferência embarcada, a decisão produzida pelo modelo não é um fim em si mesma. Ela atua como um gatilho lógico que orienta o comportamento do sistema. Quando o modelo identifica uma condição específica, como a presença de movimento, o firmware executa uma ação previamente definida, permitindo que o dispositivo reaja de forma imediata e autônoma ao evento detectado.

Essa ação pode assumir diferentes formas, dependendo da aplicação e dos periféricos disponíveis. Em sistemas simples, a resposta pode ser a ativação de um LED, o envio de uma mensagem via interface serial ou o registro de um evento em memória. Em aplicações mais avançadas, a decisão pode acionar relés, motores, atuadores eletromecânicos ou sistemas de comunicação sem fio, integrando o dispositivo a arquiteturas mais complexas de controle e monitoramento.

No firmware desenvolvido neste capítulo, a ação física está diretamente associada ao resultado da função de decisão do modelo. Quando o valor calculado indica a ocorrência de movimento, o sistema interrompe temporariamente a aquisição de dados, sinaliza o evento e reorganiza sua estrutura interna para iniciar um novo ciclo de monitoramento. Essa interrupção controlada evita múltiplas detecções consecutivas do mesmo evento e garante maior estabilidade no comportamento do sistema.

A separação entre inferência e ação é um aspecto importante do projeto. Embora estejam logicamente conectadas, essas etapas são implementadas de forma modular no firmware. O modelo de Machine Learning é responsável exclusivamente por classificar o estado do sistema, enquanto a lógica de atuação define como o dispositivo deve responder a essa classificação. Essa separação facilita a adaptação do sistema a diferentes cenários, permitindo que novas ações sejam implementadas sem a necessidade de alterar o modelo de inferência.

Outro ponto relevante é que a execução da ação física ocorre em tempo real, logo após a tomada de decisão. Não há dependência de comunicação externa ou processamento remoto para validar o resultado do modelo. Essa característica reforça o conceito de inteligência na borda, no qual o próprio dispositivo é capaz de perceber, decidir e agir de forma autônoma, com baixa latência e alta confiabilidade.

Após a execução da ação, o firmware prepara o sistema para continuar operando. O buffer de dados é reinicializado ou ajustado, o temporizador de aquisição

é reativado e o dispositivo retorna ao estado de monitoramento contínuo. Esse comportamento cíclico garante que o sistema permaneça ativo e responsivo ao longo do tempo, mesmo em ambientes dinâmicos.

Do ponto de vista didático e de engenharia, essa etapa evidencia a importância de se projetar sistemas embarcados de forma integrada. A eficácia da ação física depende diretamente da qualidade da aquisição de dados, da escolha das features, da adequação do modelo e da robustez da implementação no firmware. Todas essas camadas atuam de forma conjunta para produzir um sistema funcional e confiável.

Ao finalizar esse ciclo de operação, o dispositivo deixa de ser apenas um sensor passivo e passa a atuar como um agente inteligente no ambiente. Essa transição resume o objetivo central da implementação de Machine Learning em microcontroladores: transformar dados brutos em decisões úteis e ações concretas, executadas diretamente no ponto onde o fenômeno ocorre.

Com isso, conclui-se a implementação completa da inferência embarcada no ESP32, desde a organização do firmware e o cálculo das features até a aplicação do modelo e a execução da ação física correspondente. Esse encadeamento estabelece uma base sólida para o desenvolvimento de aplicações IoT inteligentes, reativas e eficientes, alinhadas às restrições e potencialidades dos sistemas embarcados.

## REFERÊNCIAS

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge: MIT Press, 2016.

WARDEN, P.; SITUNAYAKE, D. **TinyML**: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol: O'Reilly Media, 2020.

EMASP